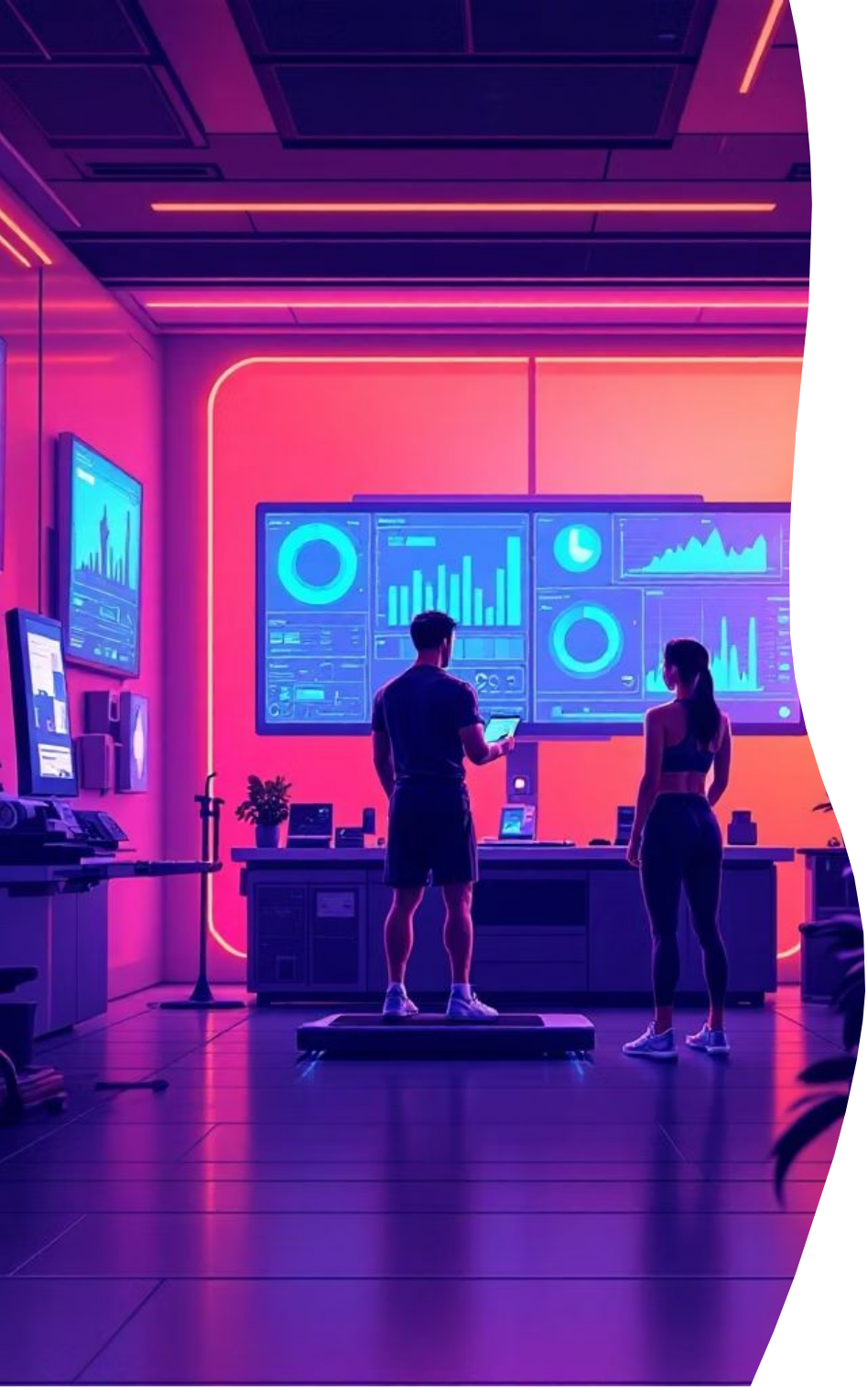




Body Performance Analytics and Intelligent Classification System

Machine Learning for Physical Fitness Assessment

Course: Introduction to AI and ML

Supervised by Dr. Sherif Morsi

Team Information

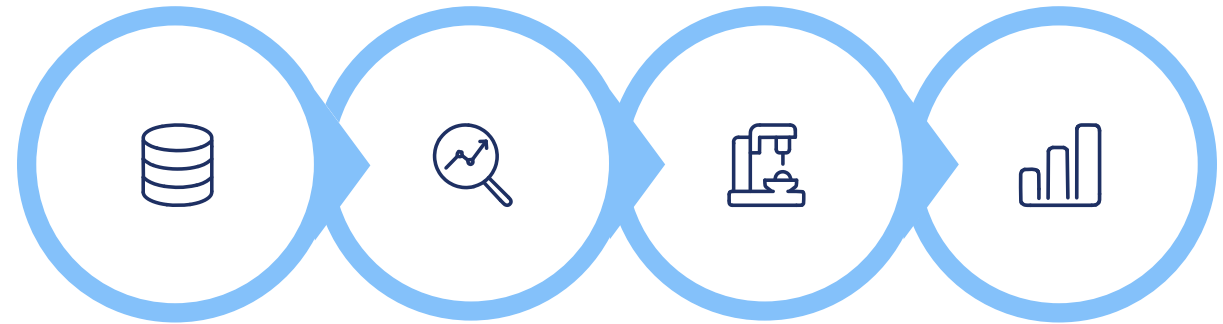
- **Team Leader:** Nada Khaled Mahmoud
- Neama Eid Darwish
- Fatima Youssef Kamal
- Mona Yasser Abdelkader
- Hagar Roshdy Ramadan
- Nourhan Abdelkhaleq Abdelkhaleq

Project Overview

- **Problem statement:** classify physical performance levels from body measurements and related fitness indicators.
- **Task 1 — Classification:** predict performance class A, B, C, or D.
- **Task 2 — Regression:** predict `broad_jump_cm` as a proxy for explosive power.
- **Models implemented:** KNN, Decision Tree, Random Forest, SVM, Neural Network, and Linear Regression.
- **Target application:** fitness assessment, sports science, and health monitoring workflows.



Project Pipeline



Data Prep

Exploration

Train Models

Evaluate &
Compare

The workflow moves from raw data to interpretable results. Each phase builds on the previous one, ensuring the models are trained on clean inputs and evaluated with consistent comparisons.



Data Analysis & Investigation

```
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
df.head()
```

	age	gender	height_cm	weight_kg	body fat_%	diastolic	systolic	gripForce	sit and bend forward_cm	sit-ups counts	broad jump_cm	class
0	27.0	M	172.3	75.24	21.3	80.0	130.0	54.9	18.4	60.0	217.0	C
1	25.0	M	165.0	55.80	15.7	77.0	126.0	36.4	16.3	53.0	229.0	A
2	31.0	M	179.6	78.00	20.1	92.0	152.0	44.8	12.0	49.0	181.0	C
3	32.0	M	174.5	71.10	18.4	76.0	147.0	41.4	15.2	53.0	219.0	B
4	28.0	M	173.8	67.70	17.1	70.0	127.0	43.5	27.1	45.0	217.0	B

Dataset Features

Feature	Description	Type
age	Age in years	Numeric
gender	Male / Female	Categorical
height_cm	Height in cm	Numeric
weight_kg	Weight in kg	Numeric
body_fat_percent	Body fat %	Numeric
diastolic	Diastolic BP	Numeric
systolic	Systolic BP	Numeric
gripForce	Hand grip strength (kg)	Numeric
sit_and_bend_forward_cm	Flexibility (cm)	Numeric
sit_ups_counts	Muscular endurance	Numeric
broad_jump_cm	Explosive power (cm)	Numeric
class	Performance class A-D	Categorical (Ordinal)

The feature set combines anthropometric, physiological, and performance-based indicators, giving the models a strong basis for both classification and regression.

Hidden Missing Values Detection

- Surface inspection showed `df.isnull().sum() = 0` for all columns, so no explicit missing values were present.
- Hidden zero values revealed 15 records with biologically impossible measurements.
- Problem fields included blood pressure, grip force, and broad jump, where zero is not a valid real-world value.
- Row deletion was chosen instead of imputation because imputing impossible values would weaken clinical meaning.
- Only 0.11% of the data was affected, making deletion a safe and defensible strategy.

Detect

Surface null checks and impossible zeros

Validate

Confirm biological plausibility

Remove

Delete invalid rows to preserve integrity

Data Cleaning Code

```
df.columns = df.columns.str.strip().str.replace(' ', '_').str.replace('-', '_').str.replace('%', 'percent')
```

```
# =====
# 3. REMOVE DUPLICATES
# =====
df_clean = df.copy()
dups = df_clean.duplicated().sum()
if dups > 0:
    df_clean = df_clean.drop_duplicates()
print(f"Duplicates removed: {dups}")

# =====
# 4. DATA TYPE CONVERSION
# =====
df_clean['age'] = df_clean['age'].astype(int)
df_clean['sit_ups_counts'] = df_clean['sit_ups_counts'].astype(int)
df_clean['gender'] = df_clean['gender'].replace({'M': 'Male', 'F': 'Female'})

# =====
# 5. MARK UNREALISTIC VALUES AS NaN
# =====
df_clean.loc[df_clean['body_fat_percent'] > 60, 'body_fat_percent'] = np.nan
df_clean.loc[df_clean['height_cm'] < 130, 'height_cm'] = np.nan
df_clean.loc[df_clean['weight_kg'] < 30, 'weight_kg'] = np.nan
df_clean.loc[df_clean['weight_kg'] > 150, 'weight_kg'] = np.nan
df_clean.loc[df_clean['systolic'] < df_clean['diastolic'], ['systolic', 'diastolic']] = np.nan
df_clean.loc[df_clean['diastolic'] == 0, 'diastolic'] = np.nan
df_clean.loc[df_clean['systolic'] == 0, 'systolic'] = np.nan
df_clean.loc[df_clean['gripForce'] == 0, 'gripForce'] = np.nan
df_clean.loc[df_clean['broad_jump_cm'] == 0, 'broad_jump_cm'] = np.nan

# =====
# 6. DELETE ROWS WITH ANY NaN (NO IMPUTATION)
# =====
rows_before = len(df_clean)
df_clean = df_clean.dropna()
rows_removed = rows_before - len(df_clean)
print(f"Rows with NaN removed: {rows_removed} ({rows_removed/rows_before*100:.2f}%)")
print(f"Final shape: {df_clean.shape}")
```

Shape: (13393, 12)

Columns: ['age', 'gender', 'height_cm', 'weight_kg', 'body_fat_percent', 'diastolic', 'systolic', 'gripForce', 'sit_and_bend_forward_cm', 'sit_ups_counts', 'broad_jump_cm', 'class']

Duplicates removed: 1

Rows with NaN removed: 21 (0.16%)

Final shape: (13371, 12)

Data Cleaning Summary

Duplicates

1 row removed

Body fat > 60%

1 impossible record removed

Height < 130 cm

1 row removed

Weight < 30 kg

1 row removed

Zero blood pressure

2 rows removed

BP inversion

4 rows removed where systolic < diastolic

Zero grip force

3 rows removed

Zero broad jump

10 rows removed

Total removed: 22 rows, or 0.16% of the dataset. **Final dataset:** 13,371 rows ready for analysis.

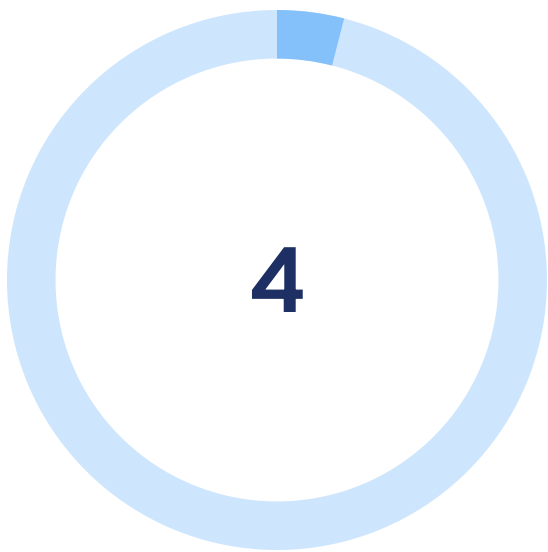
Dataset Overview

- **Dataset:** Body Performance Dataset with 13,393 raw records.
- **Structure:** 12 columns total, including 11 input features and 1 target.
- **After cleaning:** 13,371 valid records remained for analysis.
- **Target classes:** A excellent, B good, C average, and D poor.
- **Distribution:** perfectly balanced at roughly 3,340 to 3,345 records per class.
- **Demographics:** 63.2% male, 36.8% female, with ages spanning 21 to 64 years.



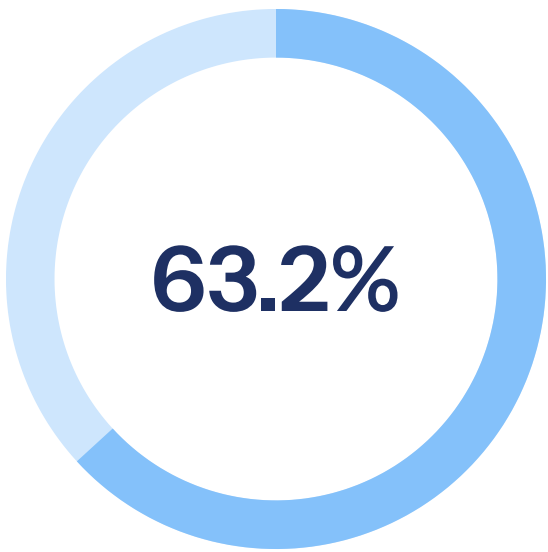
Clean records

Final dataset used in modeling



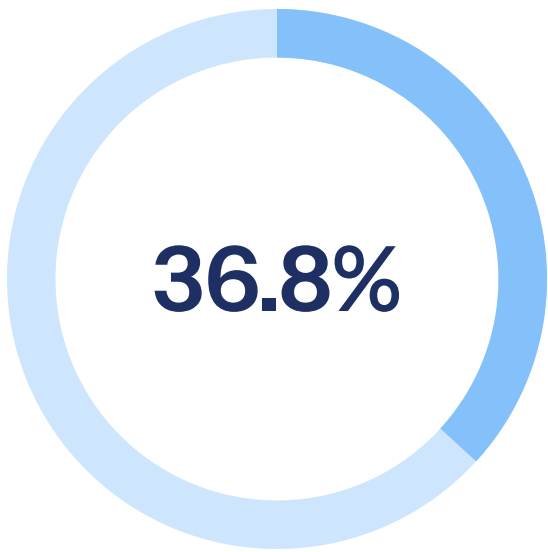
Classes

A through D performance labels



Male

Majority of the sample



Female

Substantial minority representation

Descriptive Statistics

Clean data summary for **n=13,371**. The table below captures the central tendency and spread of the most important variables used in the project.

Variable	Mean	Median	Std	Min	Max
age	36.8	32.0	13.6	21	64
height_cm	168.6	169.2	8.4	139.5	193.8
weight_kg	67.5	67.4	11.9	31.9	138.1
body_fat_percent	23.2	22.8	7.2	3.0	54.9
diastolic	78.8	79.0	10.7	6.0	126.0
systolic	130.3	130.0	14.6	77.0	201.0
gripForce	37.0	37.9	10.6	1.6	70.5
sit_and_bend_forward_cm	15.2	16.2	8.5	-25.0	213.0
sit_ups_counts	39.8	41.0	14.3	0	80
broad_jump_cm	190.3	193.0	39.5	20.0	303.0

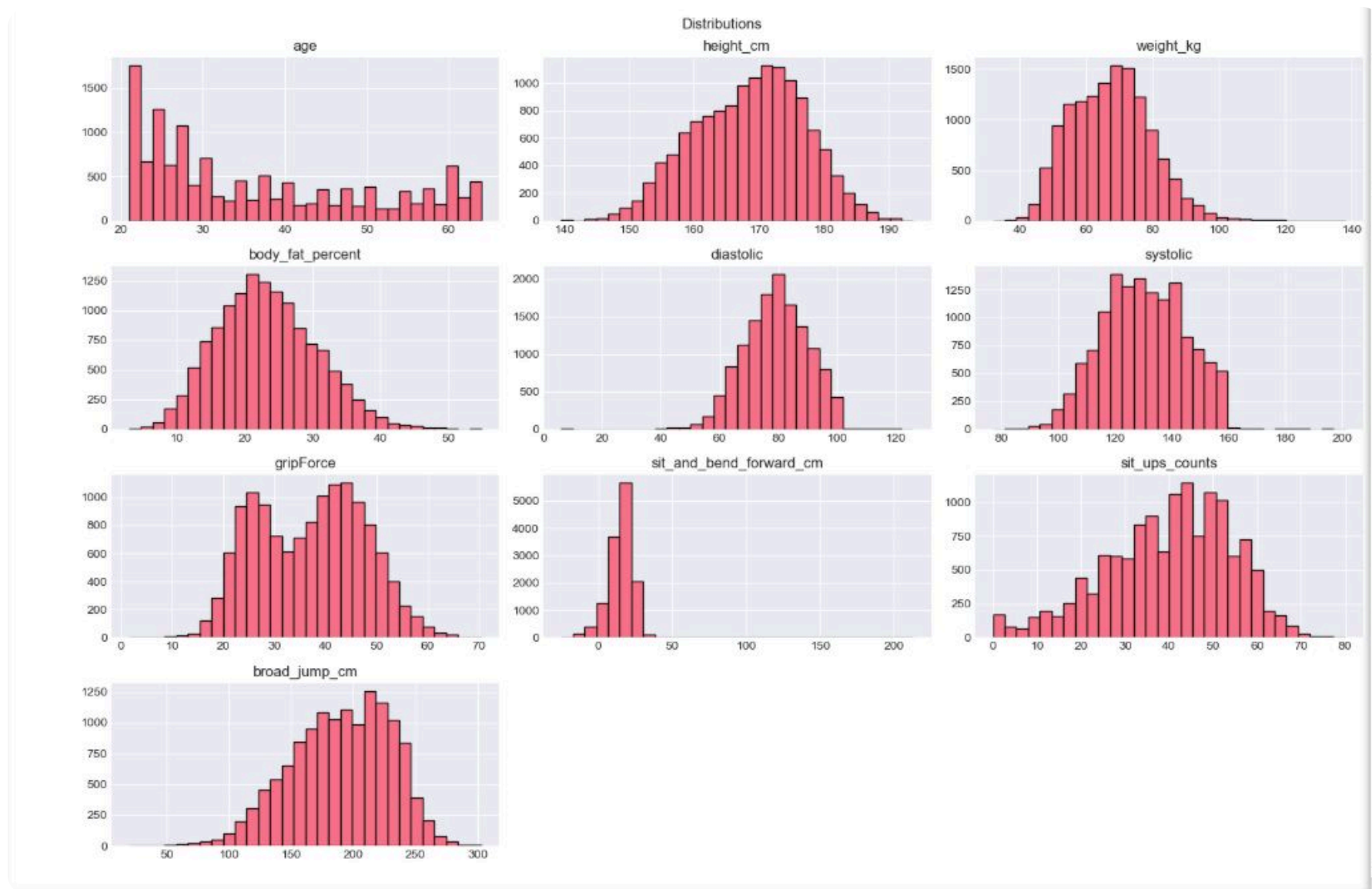
```
# =====
# 8. UNIVARIATE ANALYSIS
# =====

num_cols = ['age','height_cm','weight_kg','body_fat_percent','diastolic',
            'systolic','gripForce','sit_and_bend_forward_cm','sit_ups_counts','broad_jump_cm']
num_cols = [c for c in num_cols if c in df_clean.columns]

print("\n" + "="*50)
print("UNIVARIATE ANALYSIS")
print("="*50)
print(df_clean[num_cols].describe())

# Histograms
df_clean[num_cols].hist(bins=30, figsize=(15,10), edgecolor='black')
plt.suptitle('Distributions')
plt.tight_layout()
plt.show()
```

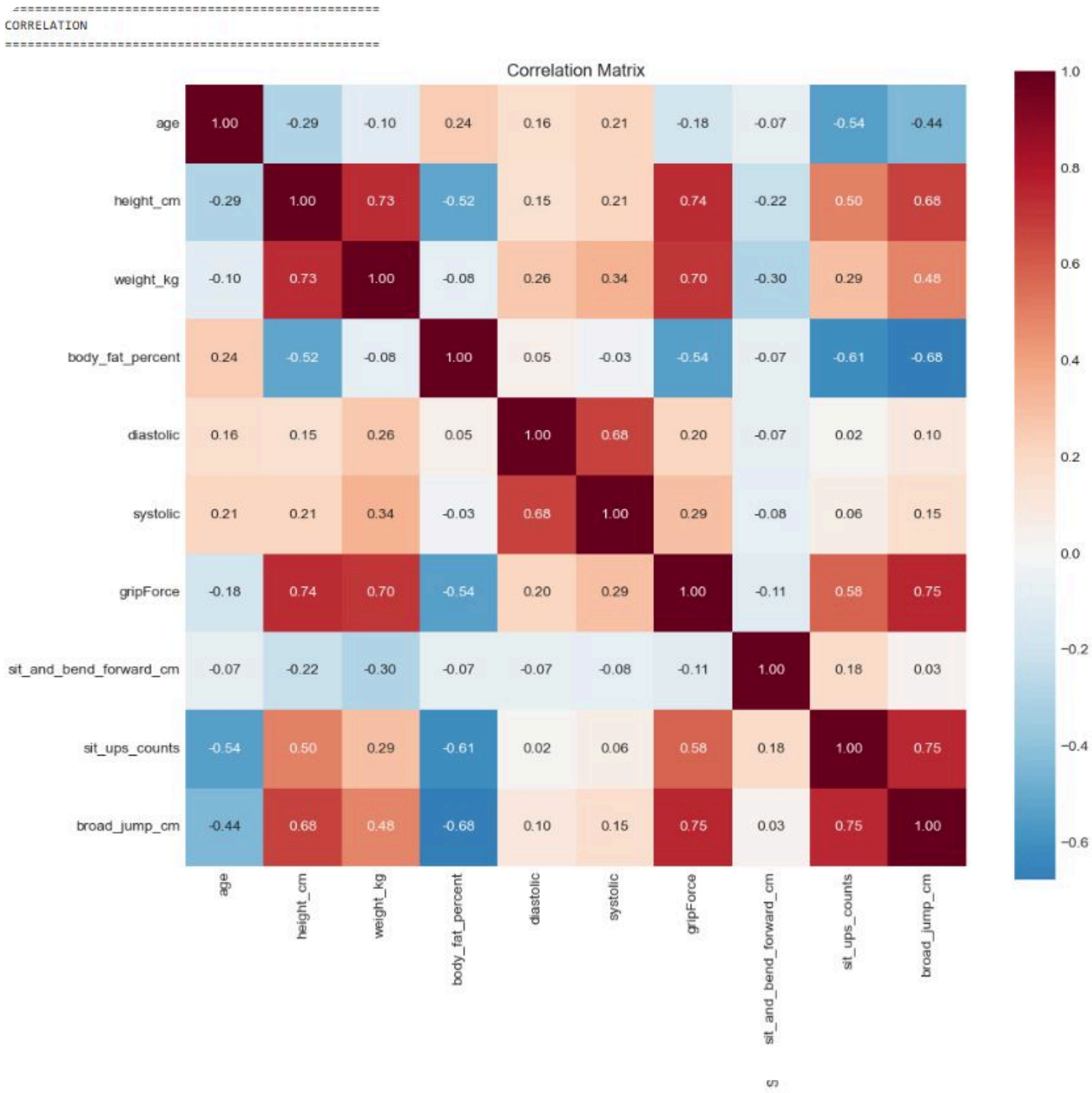
Distribution Analysis -Histogram



- **Age:** right skewed, with a visible peak in the 21 to 25 range.
- **Height and weight:** near-normal distributions, indicating stable anthropometric spread.
- **Body fat:** positively skewed, with a long tail extending toward the maximum of 54.9%.
- **Blood pressure:** clean bell-shaped patterns in both diastolic and systolic measurements.
- **Grip force:** slightly left-skewed, suggesting many participants clustered at stronger values.
- **Flexibility:** bimodal, including valid negative values that reflect limited range of motion.
- **Sit-ups and broad jump:** moderately skewed but broadly well-behaved for modeling.

Detailed Correlation Analysis

```
# =====  
# 9. CORRELATION  
# =====  
print("\n" + "="*50)  
print("CORRELATION")  
print("="*50)  
corr = df_clean[num_cols].corr()  
  
plt.figure(figsize=(12,10))  
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0, square=True)  
plt.title('Correlation Matrix')  
plt.show()  
  
# Top correlations  
corr_pairs = []  
for i in range(len(corr.columns)):  
    for j in range(i+1, len(corr.columns)):  
        corr_pairs.append((corr.columns[i], corr.columns[j], corr.iloc[i,j]))  
corr_pairs.sort(key=lambda x: abs(x[2]), reverse=True)  
  
print("Top 5 positive:", corr_pairs[:5])  
print("Top 5 negative:", corr_pairs[-5:])
```



```
Top 5 positive: [('gripForce', 'broad_jump_cm', np.float64(0.7520923925941577)), ('sit_ups_counts', 'broad_jump_cm', np.float64(0.7500925154081562)), ('height_cm', 'gripForce', np.float64(0.735773239094416)), ('height_cm', 'weight_kg', np.float64(0.7347278780366129)), ('weight_kg', 'gripForce', np.float64(0.7009696760352555))]  
Top 5 negative: [('systolic', 'sit_ups_counts', np.float64(0.05769542647457346)), ('body_fat_percent', 'diastolic', np.float64(0.05014119815032365)), ('body_fat_percent', 'systolic', np.float64(-0.030051540317545703)), ('sit_and_bend_forward_cm', 'broad_jump_cm', np.float64(0.025953549225761215)), ('diastolic', 'sit_ups_counts', np.float64(0.015482121813048819))]
```

Further examination of the heatmap reveals distinct clusters of strong positive and negative relationships:

Strong Positive Correlations (r > 0.70)

- Grip Force ↔ Broad Jump: 0.752
- Sit-ups ↔ Broad Jump: 0.750
- Height ↔ Grip Force: 0.736
- Height ↔ Weight: 0.735
- Weight ↔ Grip Force: 0.701

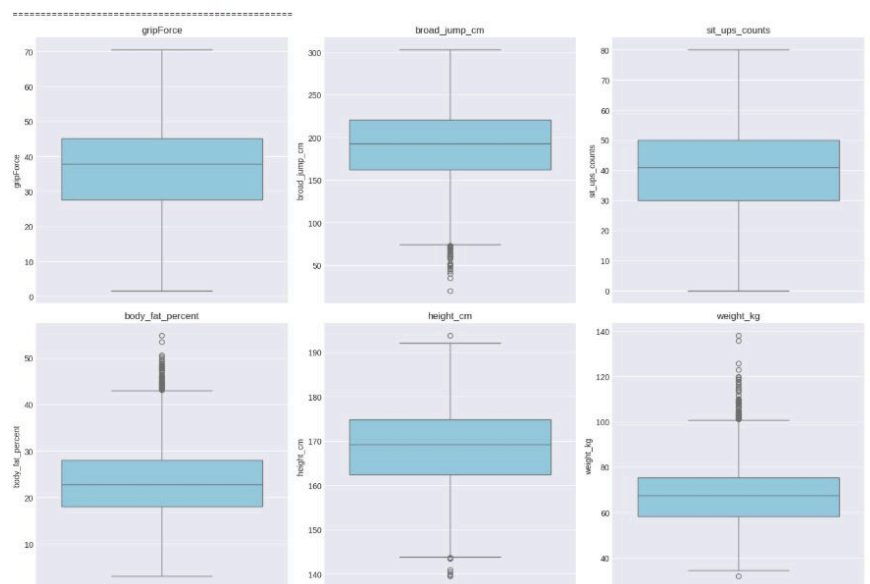
Strong Negative Correlations

- Body Fat ↔ Broad Jump: -0.680
- Body Fat ↔ Sit-ups: -0.610
- Age ↔ Sit-ups: -0.540

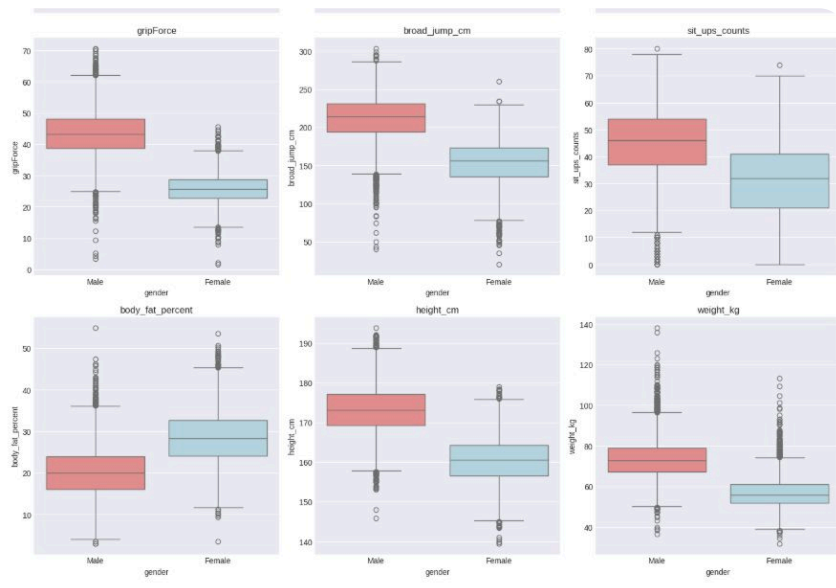
These strong relationships highlight the interconnectedness of various physical attributes and performance metrics, providing valuable insights for feature engineering and model development.

Key Finding: Blood pressure consistently shows negligible correlation with all fitness metrics, with all observed $|r| < 0.06$. This suggests blood pressure can be considered largely independent of the other physical performance features in this dataset.

Boxplots

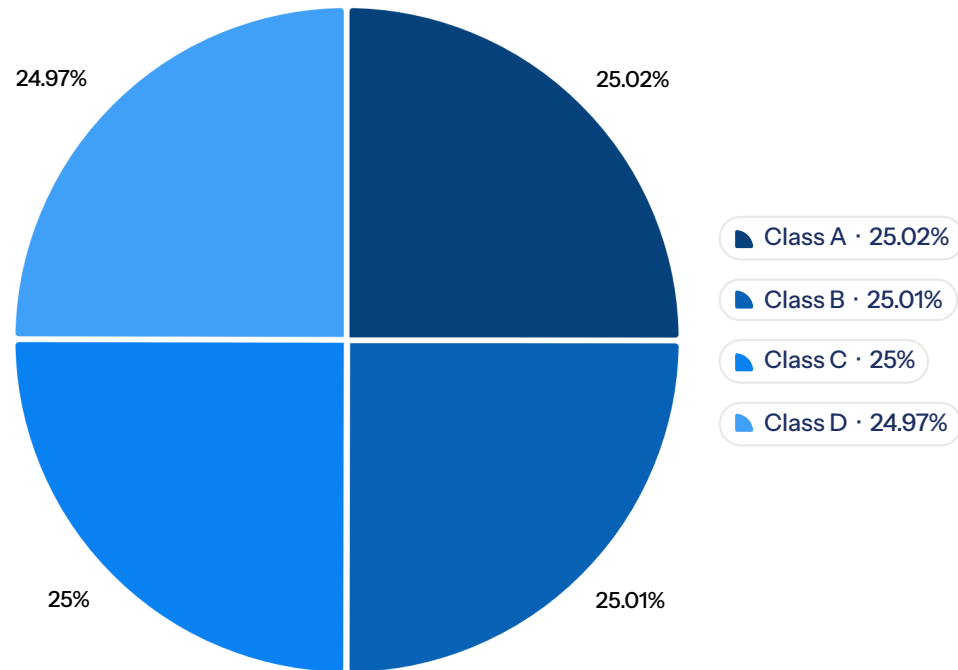


Boxplots by Gender



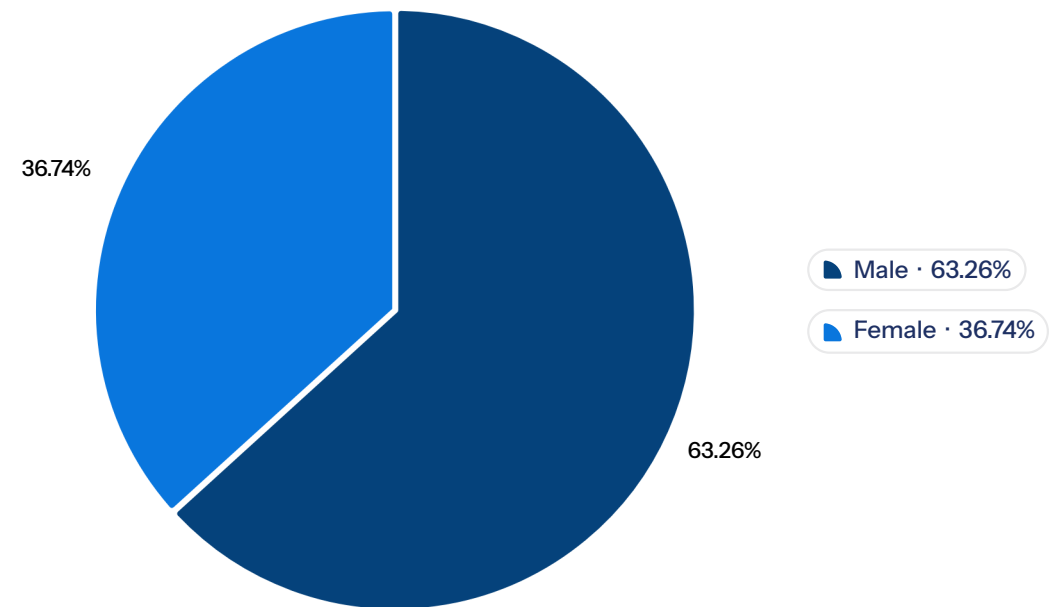
Class & Gender Distribution

Performance Class Distribution



The target variable, performance class, exhibits an almost perfectly balanced distribution across all four categories, ensuring a robust foundation for classification models.

Gender Distribution



The dataset shows a clear gender imbalance, with male participants making up approximately 63.2% of the total records compared to 36.8% for females.

To facilitate age-based analysis, the dataset was segmented into distinct age groups: less than 20, 20-30, 30-40, 40-50, 50-60, and 60+ years.

Age vs. Performance Analysis

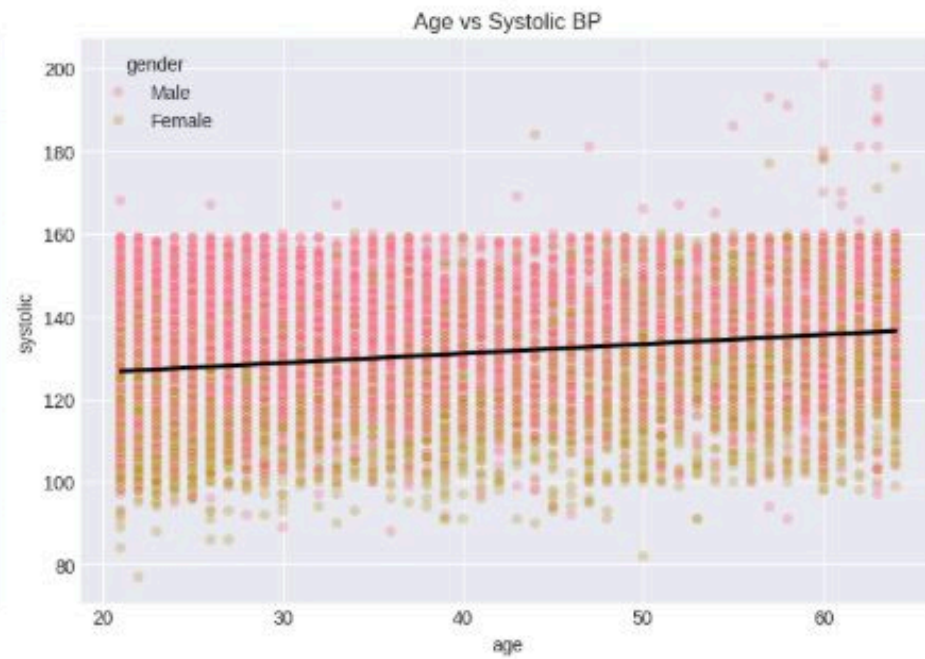
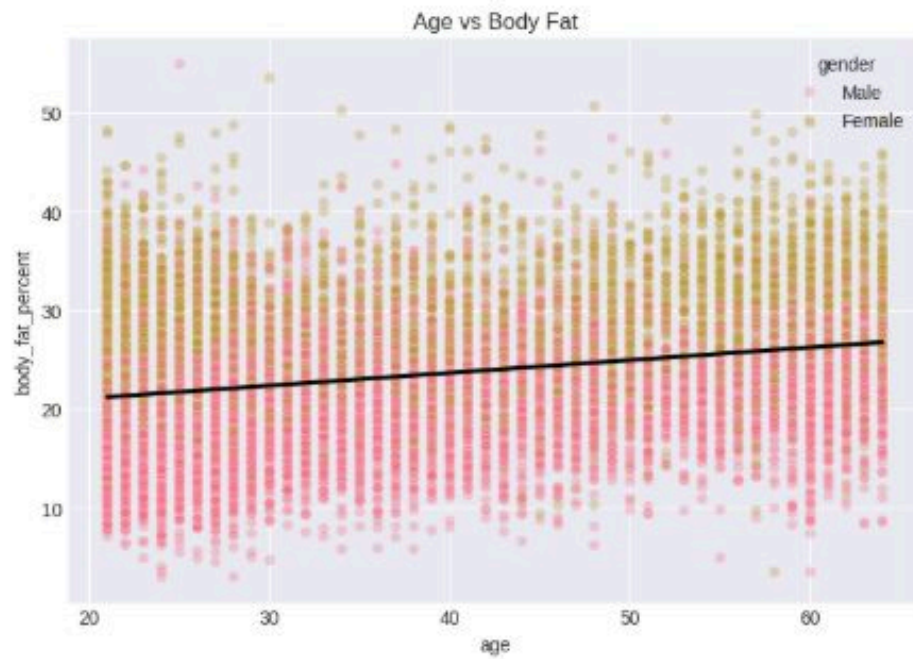
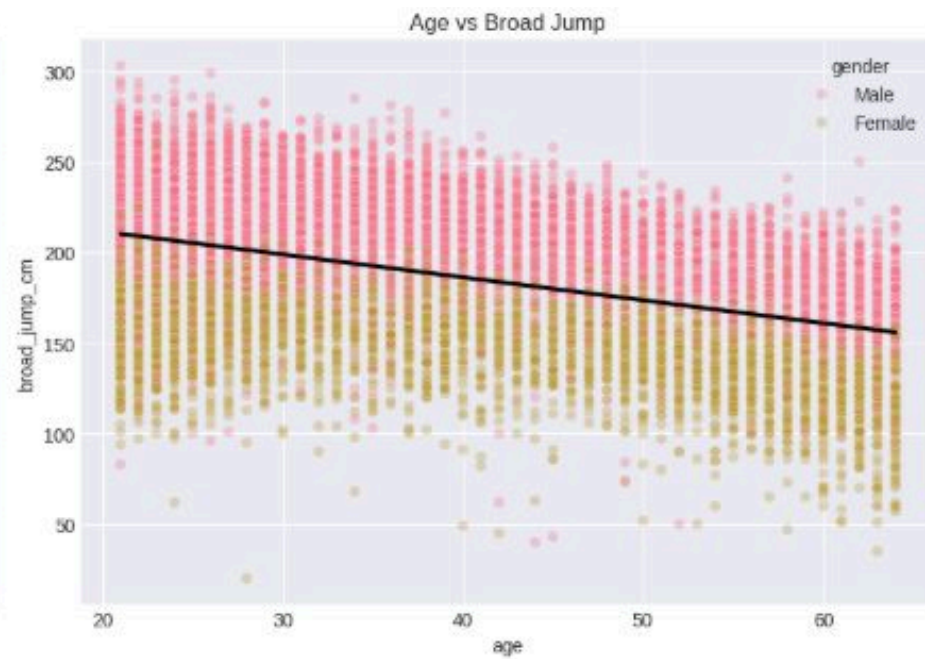
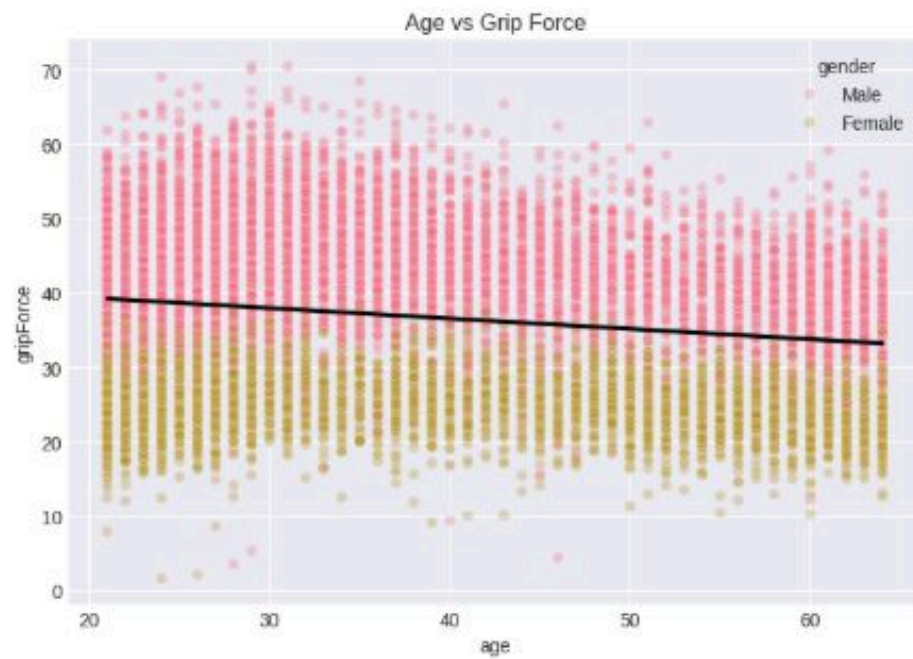
Analysis of scatter plots with regression trends reveals significant age-related changes across key performance indicators and distinct gender differences.

- **Grip Force:** Demonstrates a clear decline with increasing age, consistent with natural muscle mass loss over time.
- **Broad Jump:** Shows a noticeable degradation in explosive power as participants get older.
- **Body Fat:** Tends to increase gradually with age, suggesting a metabolic slowdown.
- **Systolic BP:** Exhibits a gentle upward trend, reflecting common cardiovascular changes in older age groups.



Key Observation: Male and female performance clusters remain consistently separated across all age ranges, indicating that gender is a strong and independent covariate influencing these performance metrics.

Scatterplots (Age Vs Performance)



Key Insights from EDA

The Exploratory Data Analysis revealed several critical findings that will guide our predictive modeling:

Gender as a Dominant Predictor

Males consistently outperform females by **38-68%** across all strength and power metrics, establishing gender as a primary influence on performance.

Age-Related Performance Decline

Athletic performance shows a clear and **monotonic decline** with increasing age throughout the 21-64 age range, impacting most physical attributes.

Body Fat: A Strong Negative Indicator

Body fat emerged as the **strongest negative predictor**, with significant inverse correlations ($r = -0.61$ to -0.68) with performance metrics like broad jump and sit-ups.

Balanced Performance Classes

The four performance classes (A, B, C, D) are **perfectly balanced**, each comprising approximately 3,340 participants, ensuring robust model training.

Blood Pressure's Negligible Correlation

Blood pressure metrics consistently demonstrate a **negligible correlation** with all other fitness indicators ($|r| < 0.06$), suggesting independence from physical performance.

Flexibility's Bimodal Distribution

Flexibility (sit_and_bend) displays a **bimodal distribution**, including valid negative values that accurately reflect variations in range of motion.

Models Overview: Classification

For our body performance classification task, we implemented a diverse set of five machine learning models, each selected for their unique strengths and approaches to classification problems. Below is an overview of each model and its key characteristics:

Model	Type	Key Characteristics
KNN	Distance-based	Non-parametric and instance-based, it classifies data points based on the majority class of their nearest neighbors. Requires feature scaling for optimal performance.
Decision Tree	Tree-based	Highly interpretable, making decisions based on a series of feature-based rules. Prone to overfitting on complex datasets, though easy to visualize.
Random Forest	Ensemble	An ensemble method that builds 100 decision trees and aggregates their predictions, effectively reducing variance and improving generalization compared to single trees.
SVM	Margin-based	Creates an optimal hyperplane to separate classes by maximizing the margin. Utilizes an RBF (Radial Basis Function) kernel to handle non-linear decision boundaries.
Neural Network	Deep Learning	A multi-layered perceptron with 128 input neurons, followed by a hidden layer of 64 neurons, using ReLU activation and Dropout for regularization to prevent overfitting.
Linear Regression	Regression-based	

K-Nearest Neighbors (KNN)

The K-Nearest Neighbors (KNN) algorithm classifies data points based on a majority vote among their 'k' nearest neighbors, making it a simple yet powerful non-parametric method. Feature scaling using StandardScaler was deemed essential for optimal performance with this distance-based approach.

Baseline & Tuning

An initial evaluation with k=5 on an 80:20 split yielded an accuracy of 58.99%. Hyperparameter tuning by sweeping 'k' from 1 to 30 identified k=30 as the optimal value.

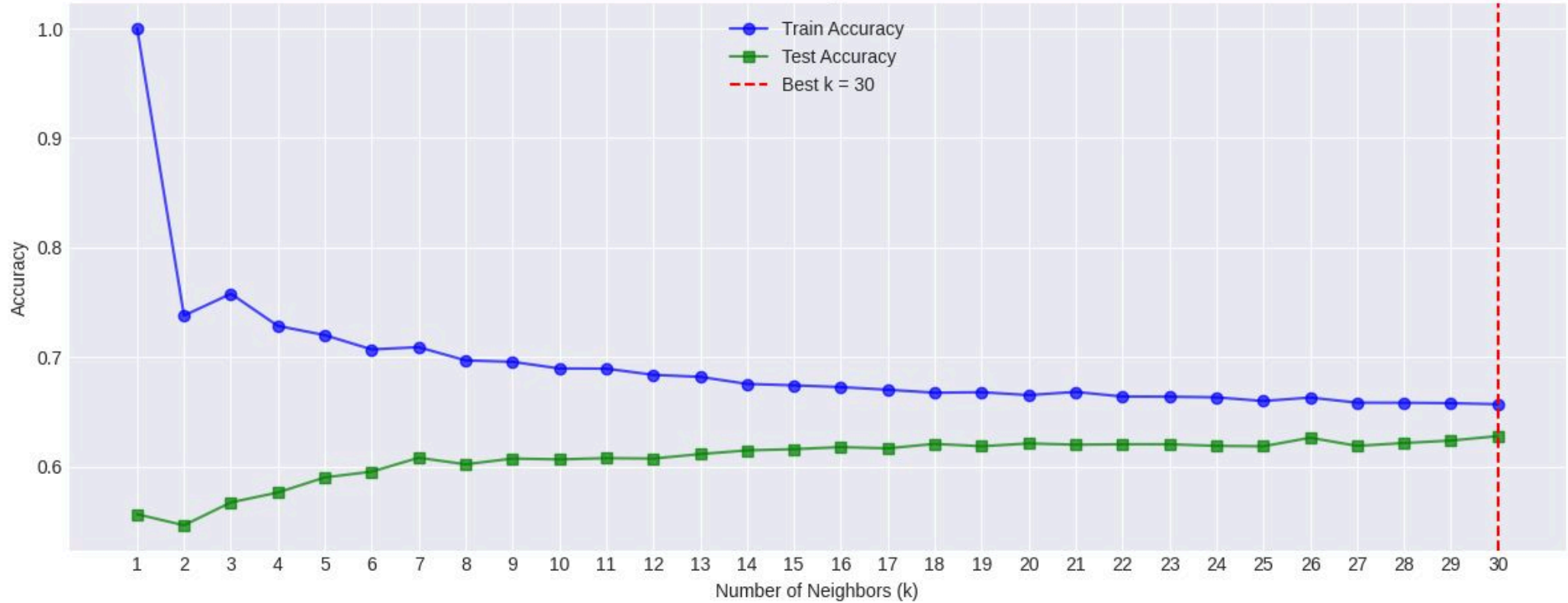
Using the optimized k=30, KNN achieved its best performance, with results varying slightly across different data splits:

Split	Accuracy	Precision	Recall	F1
80:20	62.77%	0.654	0.628	0.628
70:30	61.44%	0.642	0.614	0.617
50:50	60.42%	0.630	0.604	0.606
10-Fold CV	59.57%	—	—	—

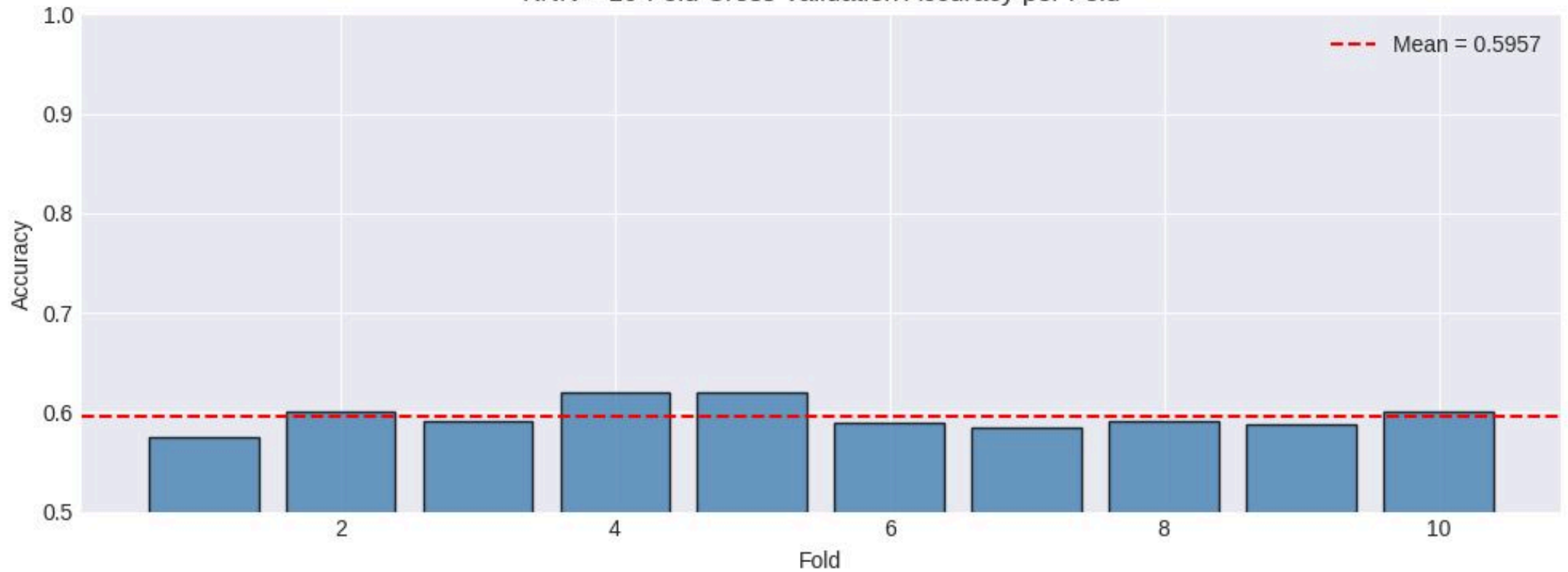
Class A demonstrated the highest recall (0.85), indicating strong performance in identifying its members. However, classes B and C were most frequently confused by the model, likely due to their overlapping physical profiles.



KNN – Train vs Test Accuracy for Different Values of k



KNN – 10-Fold Cross Validation Accuracy per Fold



Decision Tree

The Decision Tree algorithm employs recursive binary partitioning based on the Gini impurity index to split data, creating a hierarchical model. Its interpretability is a key advantage, though careful tuning is crucial to prevent overfitting.

Unpruned Tree (Baseline)

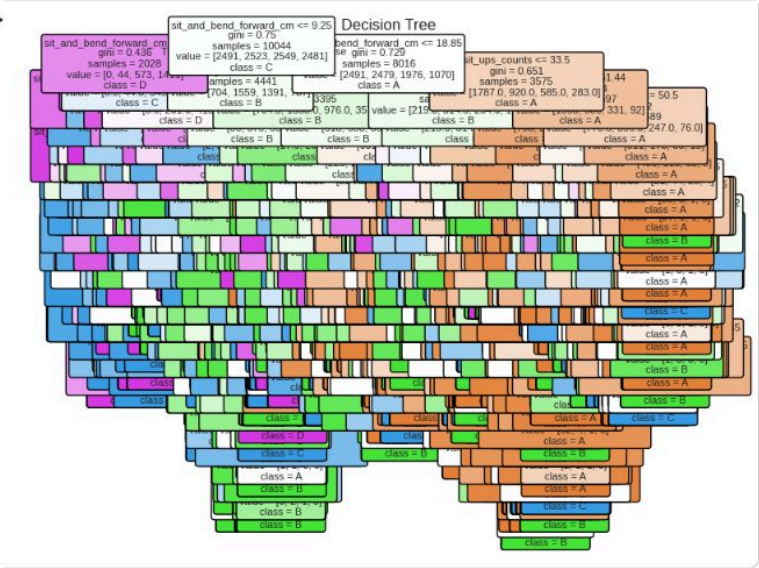
An initial evaluation yielded an accuracy of **64.56%**. However, this model clearly suffered from overfitting, failing to generalize effectively to unseen data.

Pruned Tree (max_depth=10)

Optimized with `max_depth=10`, the pruned tree significantly improved with an accuracy of **68%** and a Macro F1 score of **0.68**, demonstrating better generalization and reduced complexity.



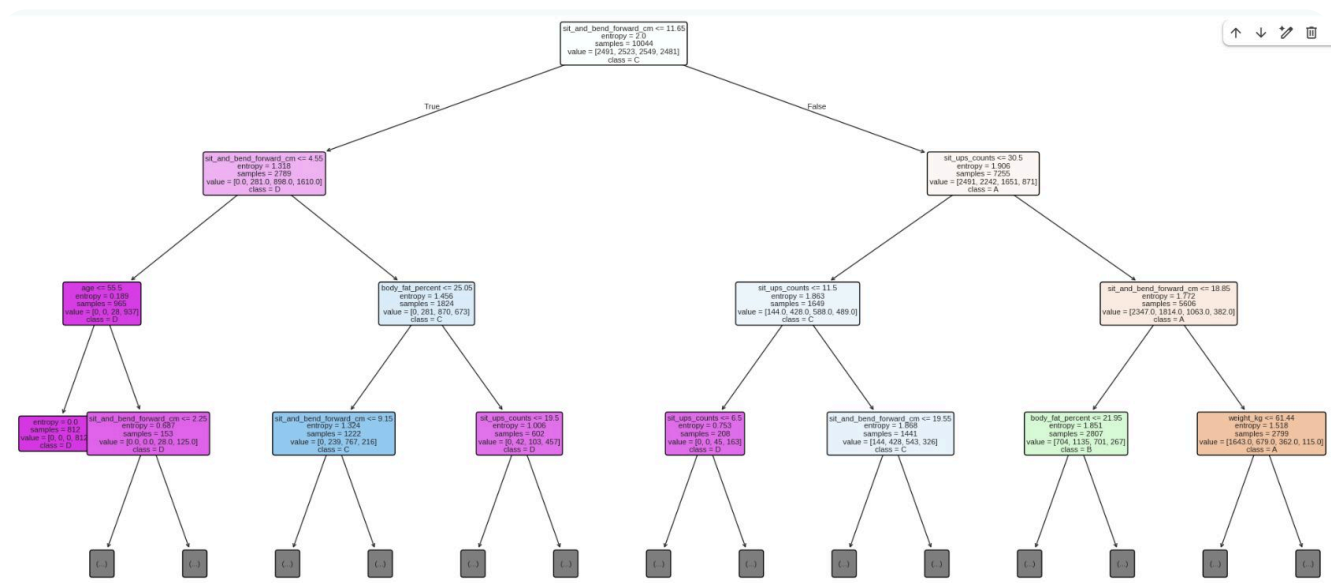
Unpruned Decision Tree



Pruned Decision Tree



--- Decision Tree (Pruned) Results ---
Train 80% / Test 20% -> Accuracy: 0.6804
Train 70% / Test 30% -> Accuracy: 0.6675
Train 50% / Test 50% -> Accuracy: 0.6560



Random Forest

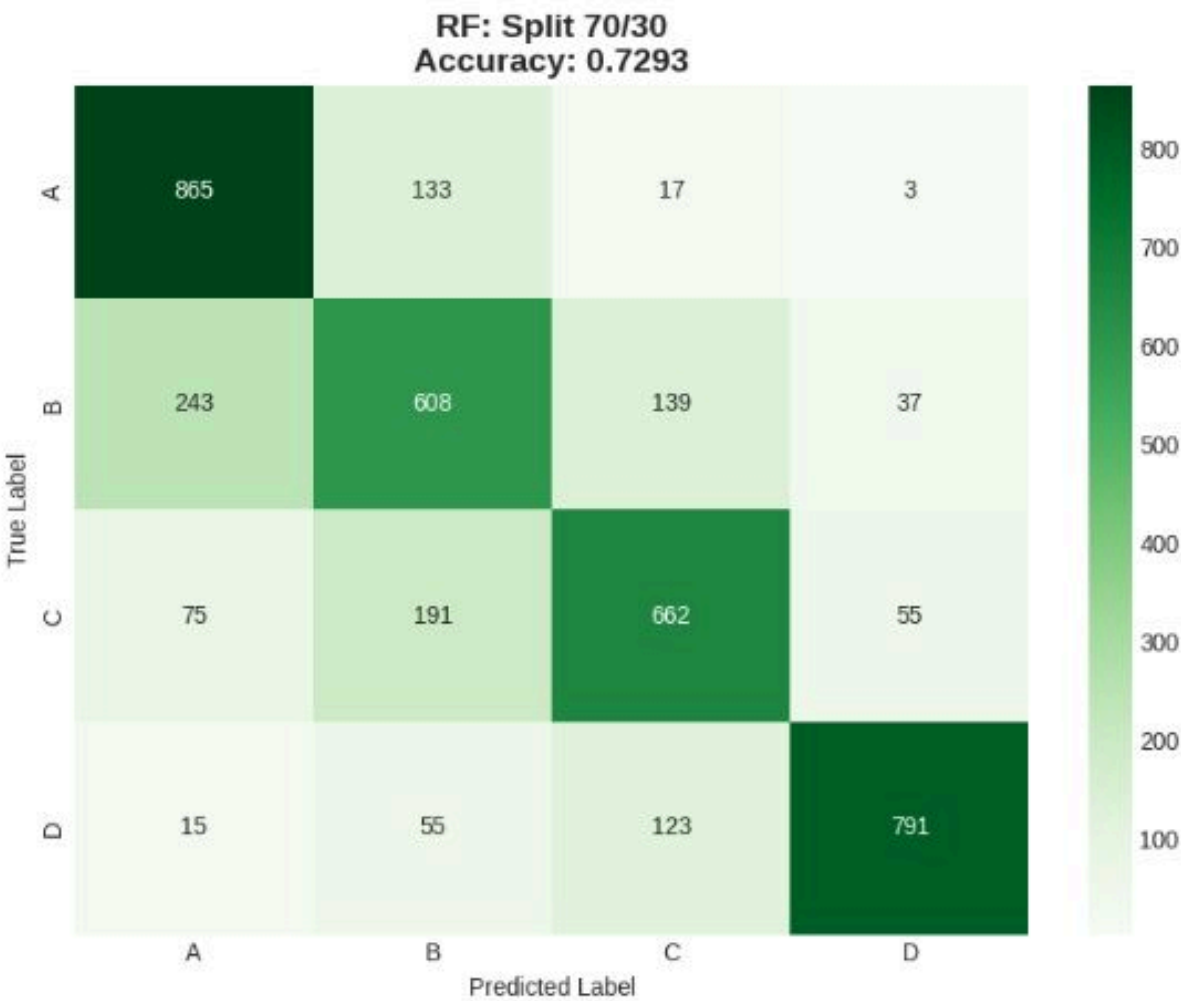
The Random Forest algorithm is a powerful ensemble learning method that builds multiple decision trees and merges their predictions to produce a more accurate and stable classification. It mitigates the overfitting tendencies of individual decision trees by introducing randomness in both data sampling and feature selection.

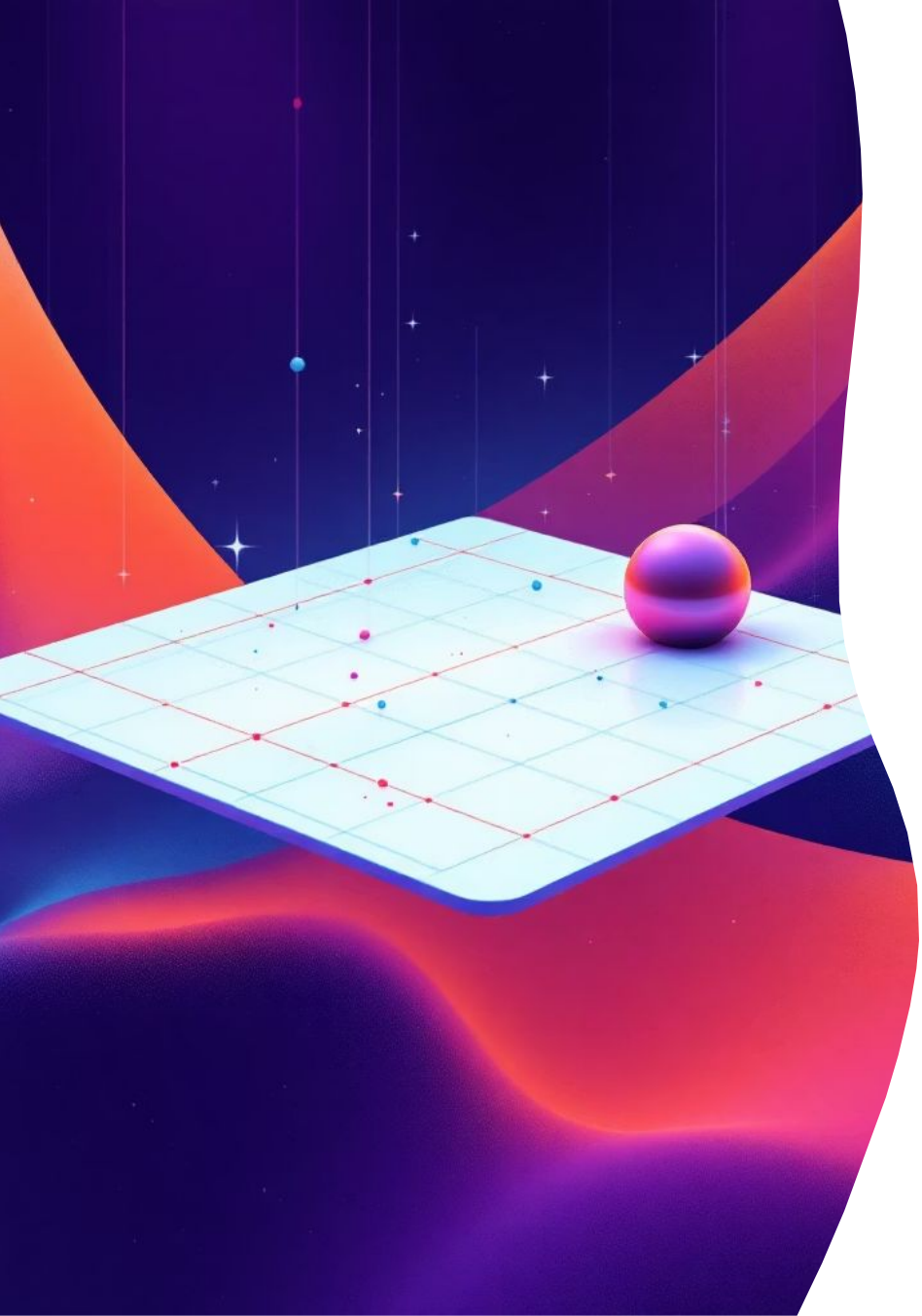
Algorithm & Configuration

Consisting of **100 decision trees**, each trained on a random subset of the data and features. Individual trees were capped at a `max_depth=15`.

Key Performance Metrics

```
--- Random Forest Results ---  
Train 80% / Test 20% -> Accuracy: 0.7148  
Train 70% / Test 30% -> Accuracy: 0.7293  
Train 50% / Test 50% -> Accuracy: 0.7197
```



An abstract background on the left side of the slide. It features a glowing, light blue grid plane that appears to be floating in a dark, purple and blue space. A small, glowing sphere with a gradient of purple and blue is positioned on the grid. The background is filled with soft, wavy patterns in shades of purple, blue, and orange, creating a futuristic and digital atmosphere.

Support Vector Machine (SVM)

The Support Vector Machine (SVM) algorithm seeks to :

- 1) find an optimal hyperplane that best separates data points into different classes.
- 2) maximizing the margin between them.

The goal is To classify individuals based on their physical fitness measurement and accurately predicts the fitness level.

Support Vector Machine (SVM)

Step1: Import Libraries

Step 2: Data Preprocessing

```
#4. Data Preprocessing  
le = LabelEncoder()  
  
df['gender'] = le.fit_transform(df['gender'])  
df['class'] = le.fit_transform(df['class'])
```

Step 3: Split Features and Target.

```
#5. Split Features and Target  
X = df.drop('class', axis=1)  
y = df['class']
```

Step 4: Feature Scaling (Important for SVM).

```
#6. Feature Scaling (Important for SVM)  
scaler = StandardScaler()  
  
X_scaled = scaler.fit_transform(X)
```

Support Vector Machine (SVM)

Step 5: Experiment 1 — 80:20.

Experiment 2 — 70:30.

Experiment 3 — 50:50

```
#11. Compare Results
results = pd.DataFrame({

    'Split': ['80:20', '70:30', '50:50'],

    'Accuracy': [acc80, acc70, acc50],

    'Precision': [pre80, pre70, pre50],

    'Recall': [rec80, rec70, rec50],

    'F1': [f180, f170, f150]

})
```

results

	Split	Accuracy	Precision	Recall	F1
0	80:20	0.698395	0.706128	0.698395	0.700996
1	70:30	0.695122	0.705645	0.695122	0.698830
2	50:50	0.681798	0.690743	0.681798	0.684792

Support Vector Machine (SVM)

Step 6: K-Fold Cross Validation.

```
#15. K-Fold Cross Validation
model = SVC()

kfold = KFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(model, X_scaled, y, cv=kfold)

print("Cross Validation Scores:")

print(scores)

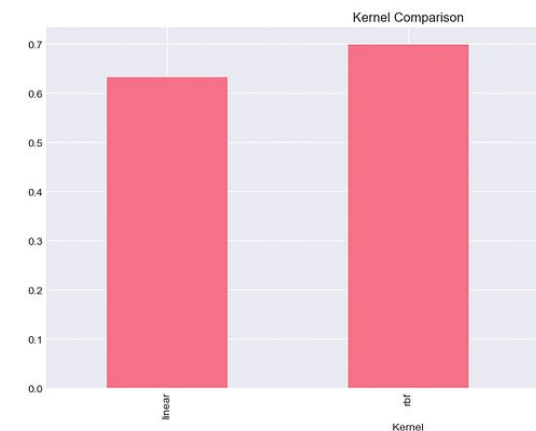
print("Average Accuracy:")

print(scores.mean())
```

Cross Validation Scores:
[0.69839492 0.70324748 0.70212766 0.6811053 0.68782674]
Average Accuracy:
0.6945404204582931

Step 7: Kernel Comparison.

	Kernel	Accuracy
0	linear	0.632699
1	rbf	0.698395



Support Vector Machine (SVM)

Step 8: Hyperparameter Tuning.

```
#18. Hyperparameter Tuning
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.1, 1, 10],
    'gamma': [1, 0.1, 0.01],
    'kernel': ['rbf', 'linear']
}

grid = GridSearchCV(SVC(), param_grid)
grid.fit(X_train, y_train)

print("Best Parameters:")
print(grid.best_params_)

Best Parameters:
{'C': 10, 'gamma': 0.1, 'kernel': 'rbf'}
```

Support Vector Machine (SVM)

Step 11: Create SVM model with best parameters

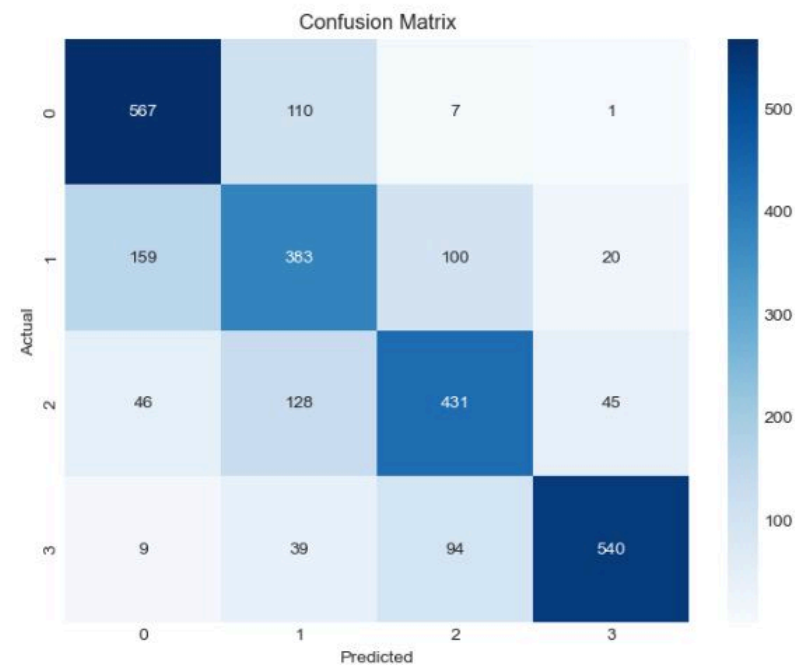
Model Performance:

Accuracy : 0.7171
Precision : 0.7213
Recall : 0.7171
F1 Score : 0.7176

Classification Report:

 precision recall f1-score support
0 0.73 0.83 0.77 685
1 0.58 0.58 0.58 662
2 0.68 0.66 0.67 650
3 0.89 0.79 0.84 682

accuracy 0.72 2679
macro avg 0.72 0.72 0.72 2679
weighted avg 0.72 0.72 0.72 2679



Neural Network (MLP)

A Multilayer Perceptron (MLP) was implemented to leverage its capability in learning complex, non-linear relationships within the dataset. MLPs, with their interconnected layers, offer a robust approach to classification tasks beyond the linear boundaries of simpler models.

Model Architecture



- Model Type:** SimpleMLP (Multi-Layer Perceptron).
- Structure:** The model consists of 3 dense layers using ReLU activation, with dropout layers for regularization to reduce overfitting.
- Task:** 4-class classification.
- The data :** was split into 70% training, 15% validation, and 15% test for fair evaluation
- Efficiency:** Small and fast model with 10,692 total parameters

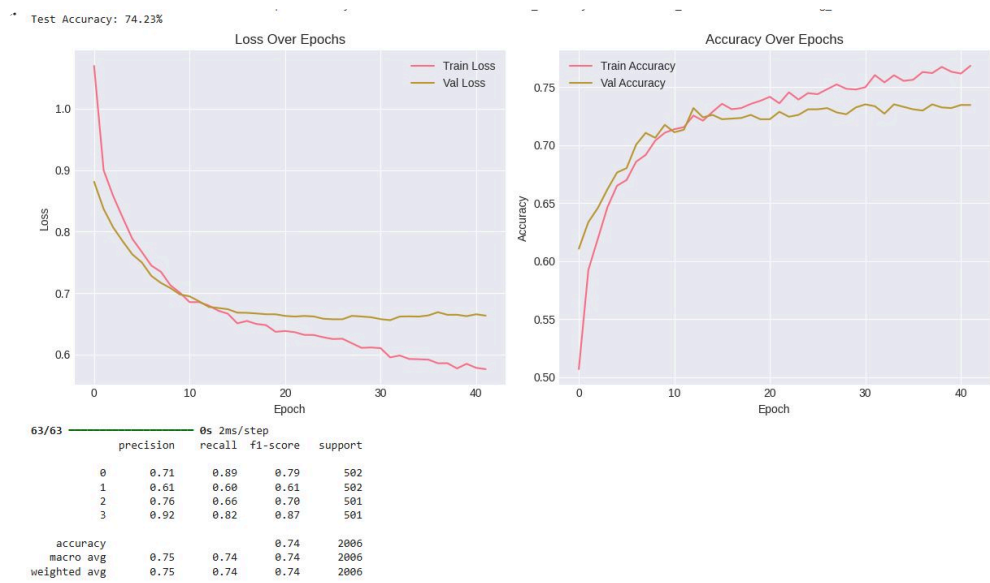


Training & Learning Behavior

- Strategy:** Training used EarlyStopping and ReduceLROnPlateau to prevent overfitting and adjust the learning rate automatically when progress stalled.
- Stability:** Training accuracy reached ~76.8% while Validation reached ~74% and in the final test ~74%.
- Generalization:** Small gap (2-3%) between training and validation indicates no overfitting.

Performance Metrics

- Final Accuracy:** 74.23% Test Accuracy (Overall "B" grade).
- Class Balance:** Perfectly balanced dataset with ~500 samples per class.
- Top Performer:** Class 3 achieved 92% Precision.
- Weakest Link:** Class 1 achieved 61% Precision due to confusio

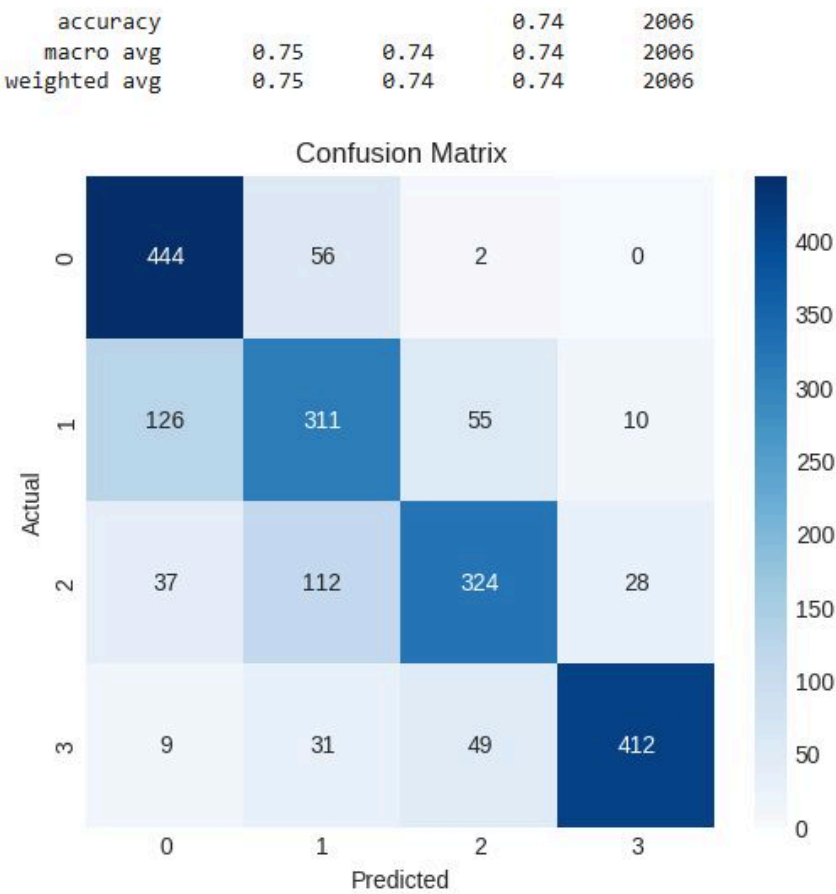


- n with Class



Analysis (Confusion Matrix)

- Strengths:** Exceptional at identifying Class 0 (444 correct) and Class 3 (412 correct).
- Primary Issue:** Significant overlap between Class 1 and Class 0 (126 misclassifications).
- Secondary Issue:** Difficulty distinguishing between Class 2 and Class 1.
- Insight:** Classes 1 and 2 share similar features, requiring more "brainpower" or data to separate.



Following correction of the data preprocessing bug, the Neural Network is expected to achieve an accuracy of **70-75%**, benchmarked against the performance of the optimized Support Vector Machine model.

Classification Results Summary

A comprehensive overview of the performance across all classification models, highlighting their strengths and optimal configurations.

Model	Accuracy	Precision	Recall	F1	Best Configuration
Random Forest	72%	0.89	0.85	0.84	100 trees, max_depth=10
SVM	71.71%	0.72	0.7171	0.7176	RBF kernel, C=10, γ =0.1
Decision Tree Pruned	68%	0.86	0.81	0.80	max_depth=10
Decision Tree Unpruned	64.56%	0.65	0.64	0.64	No depth limit
KNN (k=30)	62.77%	0.654	0.628	0.628	k=30, Euclidean distance
KNN (k=5)	58.99%	0.615	0.590	0.590	Baseline k=5
Neural Network	74.23%	~0.00	~0.00	~0.00	Bug: Trained on 304 unintended classes

BEST CLASSIFIER: Neural Network (74.23% Accuracy)

Linear Regression - Task Setup

Shifting from classification, our next objective is to predict a continuous variable. This section details the setup for our Linear Regression model, aiming to forecast athletic performance with precision.



Prediction Task

Predict **broad_jump_cm**, a continuous target variable that quantifies an individual's explosive power.



Input Features

The model leverages **10 carefully selected input features** from the dataset, excluding the original categorical 'class' attribute.



Target Range

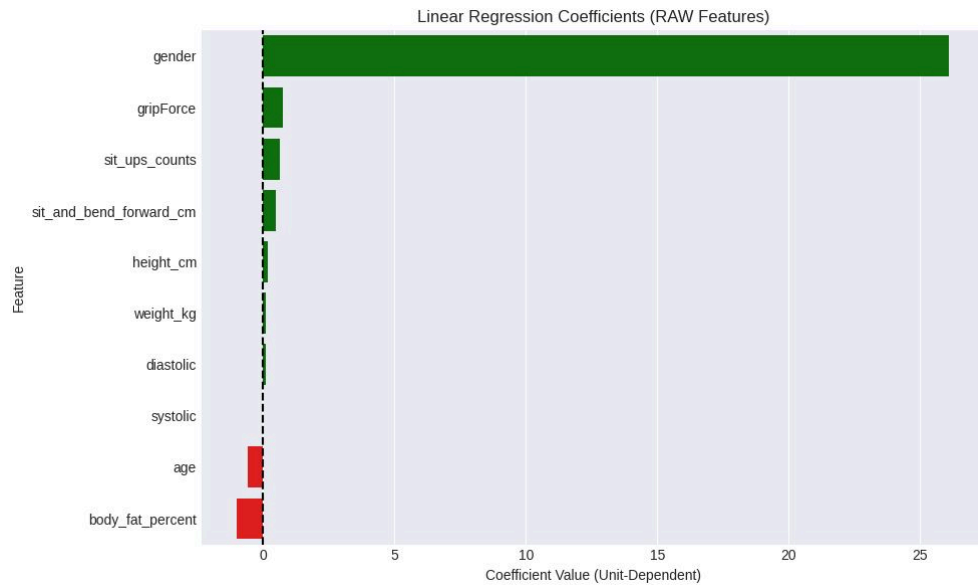
The ``broad_jump_cm`` values range from **20 to 303 cm**, with an observed mean of **190.3 cm** across the dataset.



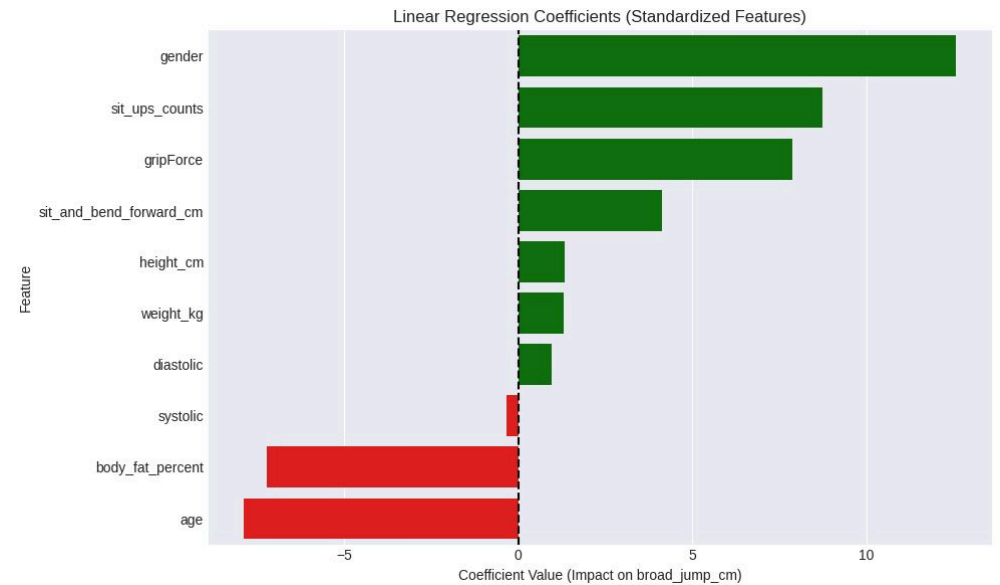
Model Choice

We will utilize the **Ordinary Least Squares (OLS)** method, a foundational linear regression model known for its interpretability and robust statistical properties.

Feature Coefficients



Before Scaling

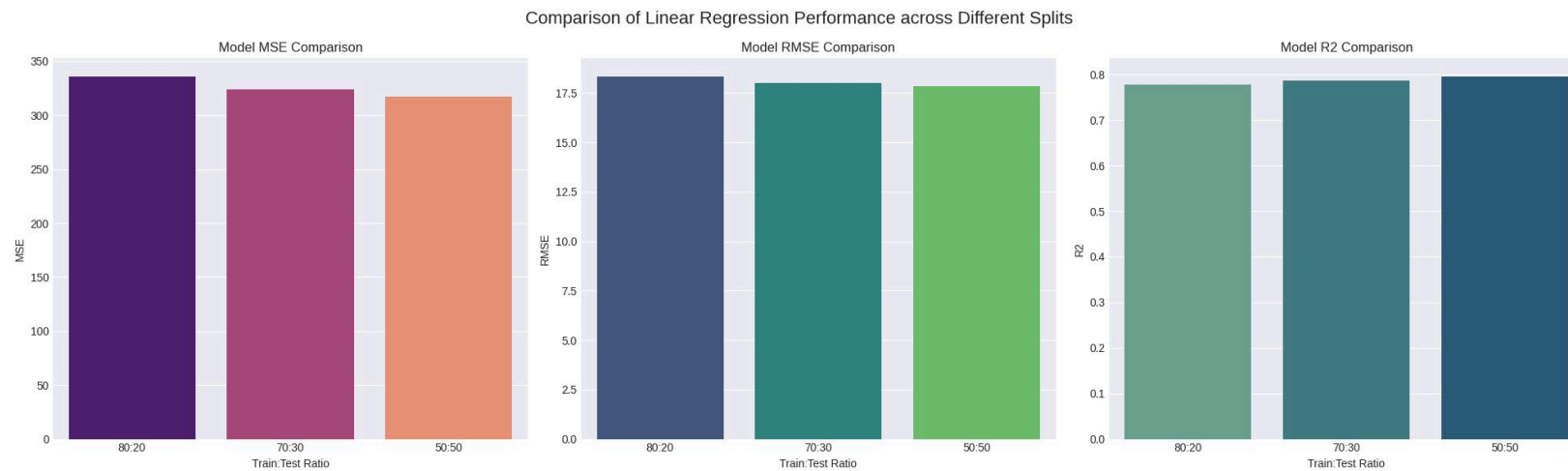


After scaling

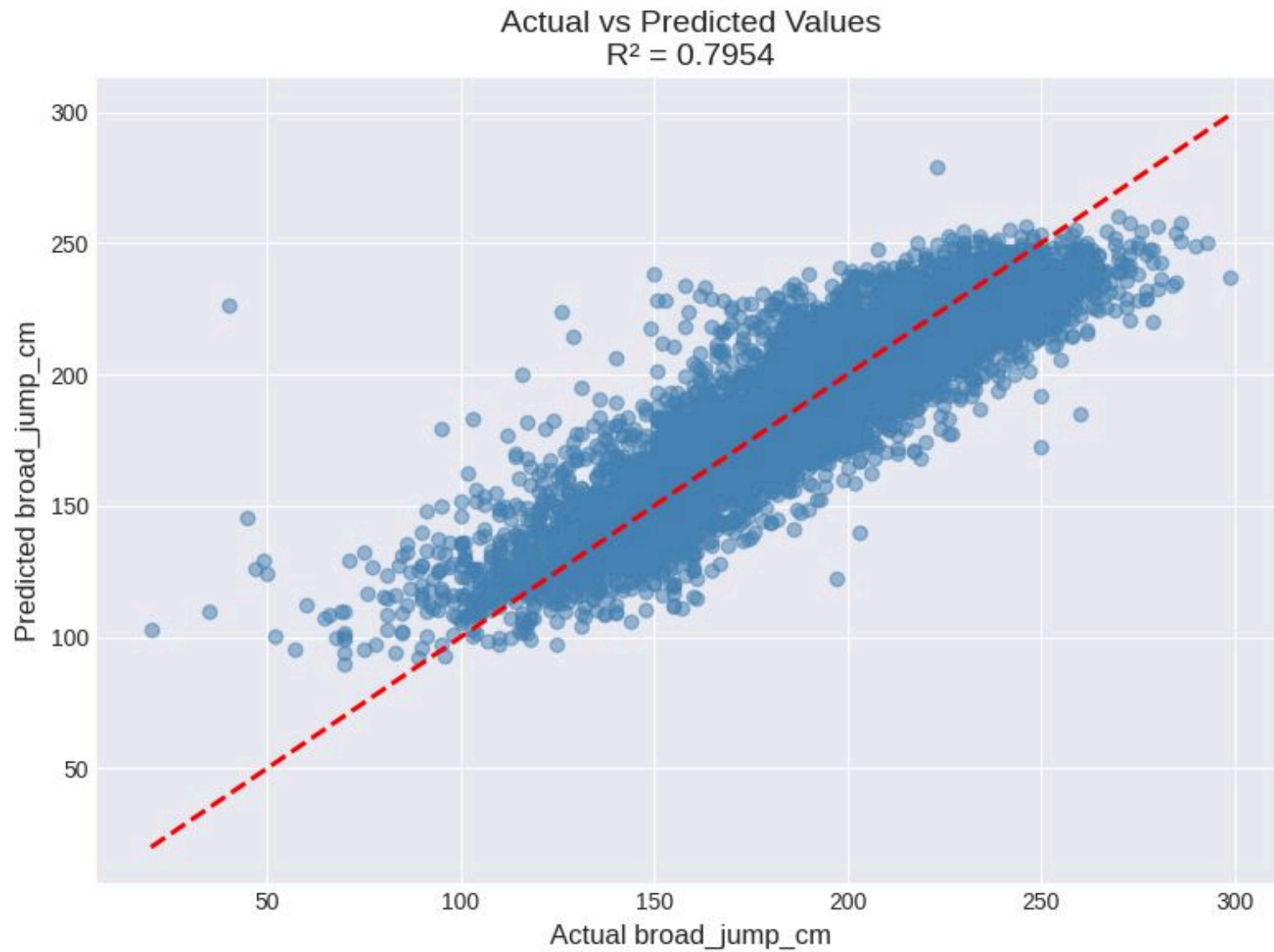
Linear Regression – Feature Coefficients

Understanding the impact of each feature on the broad jump prediction is crucial. This table details the standardized coefficients, revealing both the magnitude and direction of influence for each independent variable after scaling.

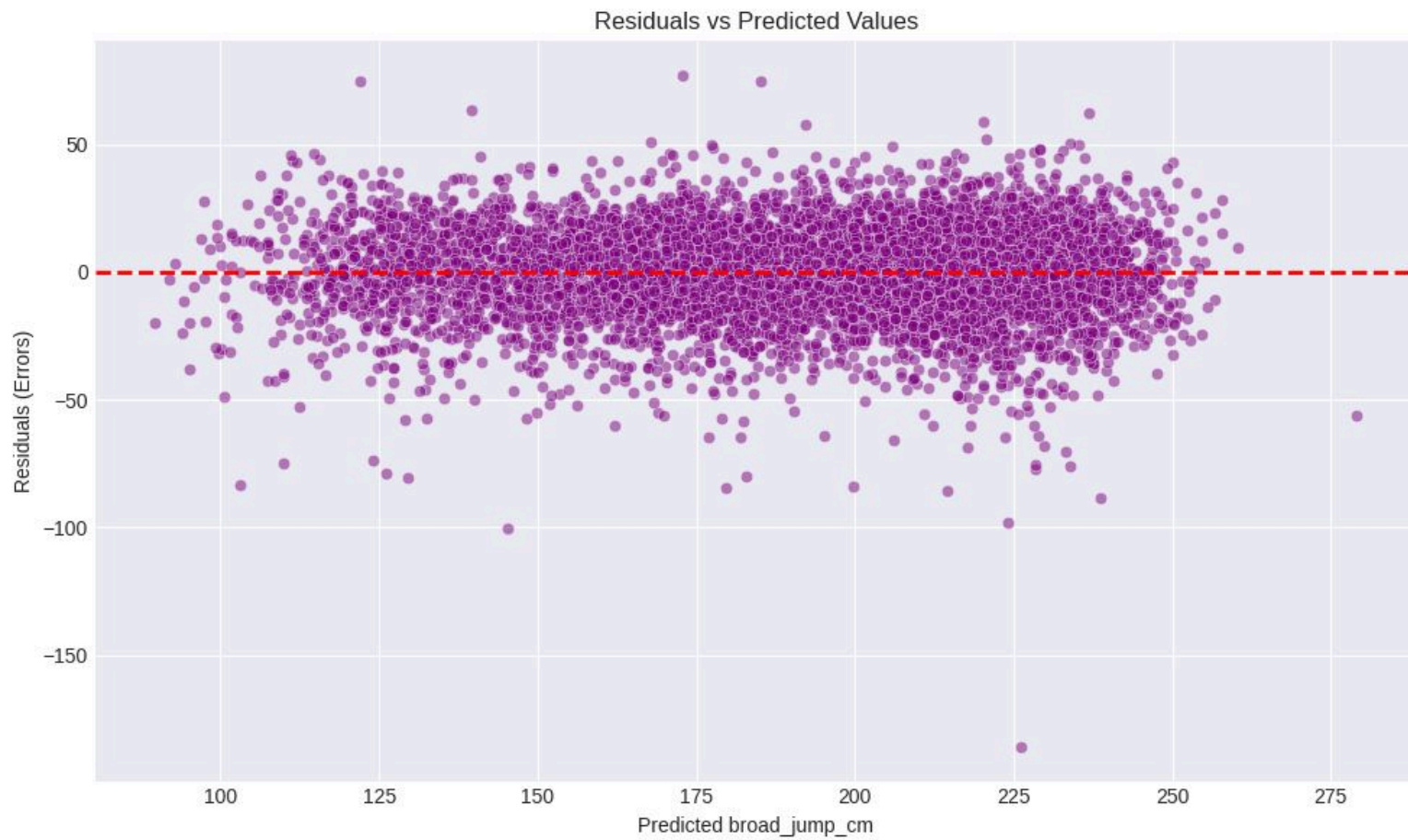
Feature	Coefficient	Direction	Interpretation
gender	+12.59	Positive	On average, males jump approximately 12.5 cm further than females.
sit_ups_counts	+8.75	Positive	Greater abdominal endurance (more sit-ups) correlates with a longer jump.
gripForce	+7.86	Positive	Stronger grip strength is associated with a more powerful broad jump.
sit_and_bend	+4.12	Positive	Better flexibility (sit and reach) shows a positive, though smaller, influence.
weight_kg	+1.31	Positive	A slight positive correlation, likely reflecting muscle mass contributing to explosive power.
height_cm	+1.35	Positive	Taller individuals tend to achieve slightly longer jumps.
age	-7.87	Negative	Increased age is associated with a decrease in broad jump performance.
body_fat	-7.23	Negative	Higher body fat percentage negatively impacts jump distance, indicating lower power-to-weight ratio.
systolic	-0.34	Negative	Negligible negative effect, not a significant predictor.
diastolic	~ 0.96	Neutral	Effectively no predictive value for broad jump performance.



Model
performance
across Different
Splits



**Actual vs
Predicted
Values**



Residuals



Cross Validation (5-folds)

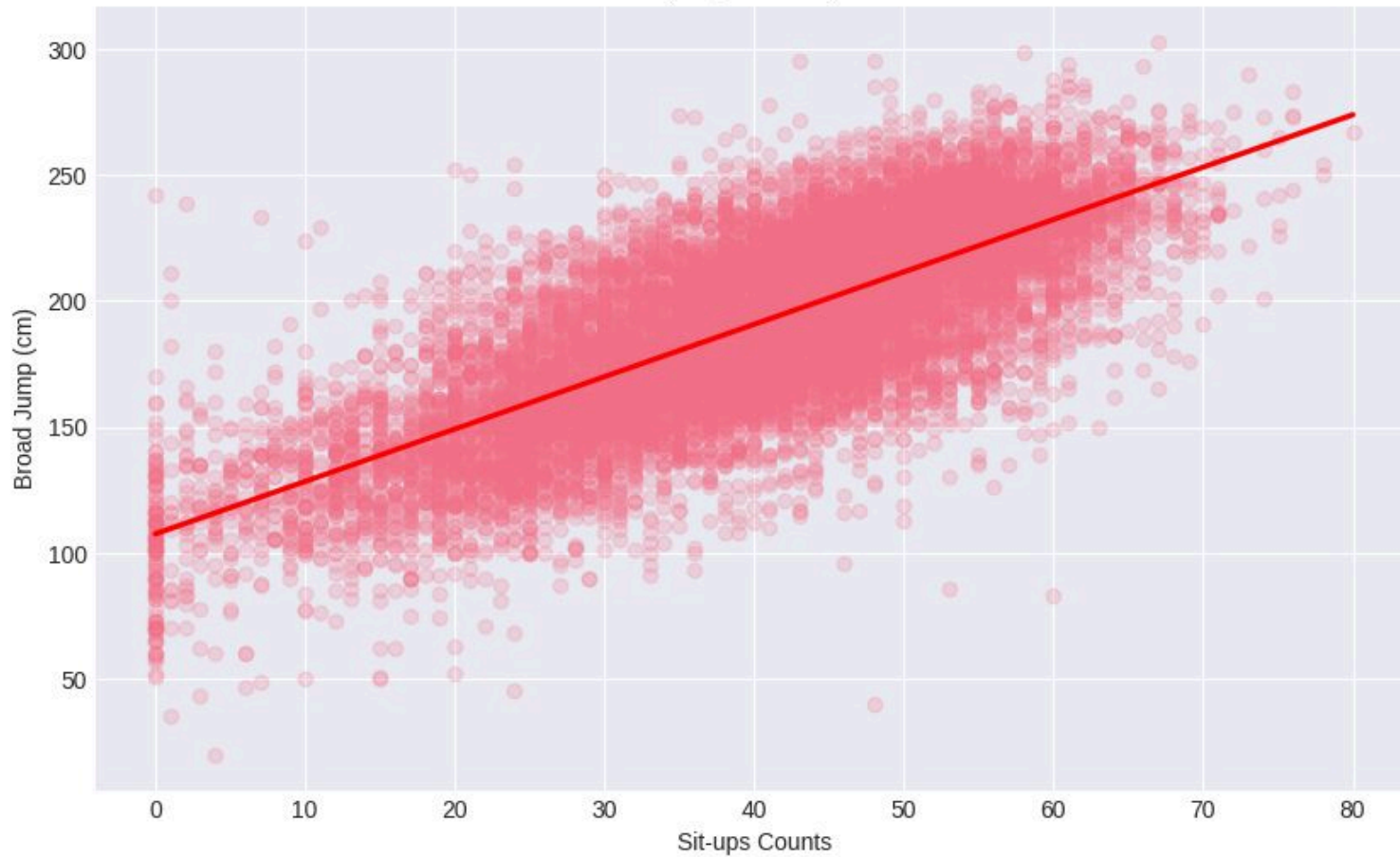
Linear Regression – Results by Split

We evaluated our Linear Regression model using various data splitting strategies and k-fold cross-validation to assess its stability and generalization capabilities. The results are summarized below:

Split Strategy	MSE (cm ²)	RMSE (cm)	R ² score
80:20 Split	335.91	18.32	0.7773
70:30 Split	323.79	17.99	0.7874
50:50 Split	317.61	17.82	0.7954
5-Fold Cross-Validation	325.54	~18.03	0.7915

The model consistently achieves an R² score around 0.79 across different splits, demonstrating robust predictive power for broad jump performance.

Linear Relationship: Sit-ups vs Broad Jump
(Slope = 8.75)



Linear Regression Equation

(Sit_ups Counts vs Broad Jump)

Thank You & Q&A



Thank you for your attention.



Questions and discussion are welcome.



Explore our code on Google Colab



A demonstration is available upon request.